DOCUMENT D'ARCHITECTURE TECHNIQUE

Billetterie — Les Écrans de Villeneuve

Équipe : à compléter | Membres : à compléter | Version : 1.0 de travail | Date : 5 novembre 2026

Document préparé à partir du sujet fourni. Les choix marqués comme hypothèses doivent être validés par l'équipe avant dépôt.

1. Introduction et objectifs

Le produit

La billetterie des Écrans de Villeneuve remplace le logiciel historique du multiplexe. Elle centralise la vente de places numérotées pour 8 salles et 1 450 fauteuils, depuis le site web, l'application mobile, quatre bornes, trois guichets et deux partenaires. Le cœur du produit garantit l'unicité de vente d'un fauteuil pour une séance, la réservation temporaire pendant le paiement, la confirmation des billets et la traçabilité des ventes.

Les utilisateurs

Utilisateur	Ce qu'il fait avec le système	Canal
Clients	Choisir, réserver, payer, annuler et recevoir les billets	Web, mobile, bornes
Caissiers	Vendre pour un client présent et encaisser	Guichets
Abonnés	Souscrire, réserver la place gratuite et annuler	Web, guichet
Agents d'accueil	Scanner les billets et contrôler l'entrée	Douchettes
Programmateur	Publier, déplacer et annuler les séances	Back-office
Marketing	Gérer tarifs, opérations et abonnements	Back-office
Responsable financier	Suivre les ventes et produire le fichier CNC	Back-office
Partenaires	Transmettre des ventes déjà payées	API partenaire

Les objectifs de qualité

Priorité	Objectif	Mesure cible proposée	Origine
1	Cohérence des ventes	0 double vente d'un même fauteuil ; conflit renvoyé en 409	B1, incident historique
2	Disponibilité de la vente	Service en ligne disponible 99,9 % sur les plages d'ouverture ; reprise après incident documentée	Vente sans interruption mercredi et week-end
3	Évolutivité / maintenabilité	Ajouter un partenaire en moins de 2 semaines et une formule en moins d'un mois	Direction générale
4	Contrôle d'accès résilient	Scan hors réseau pendant au moins 4 heures (hypothèse à valider)	B12, Karim

2. Contraintes et hypothèses

Contraintes

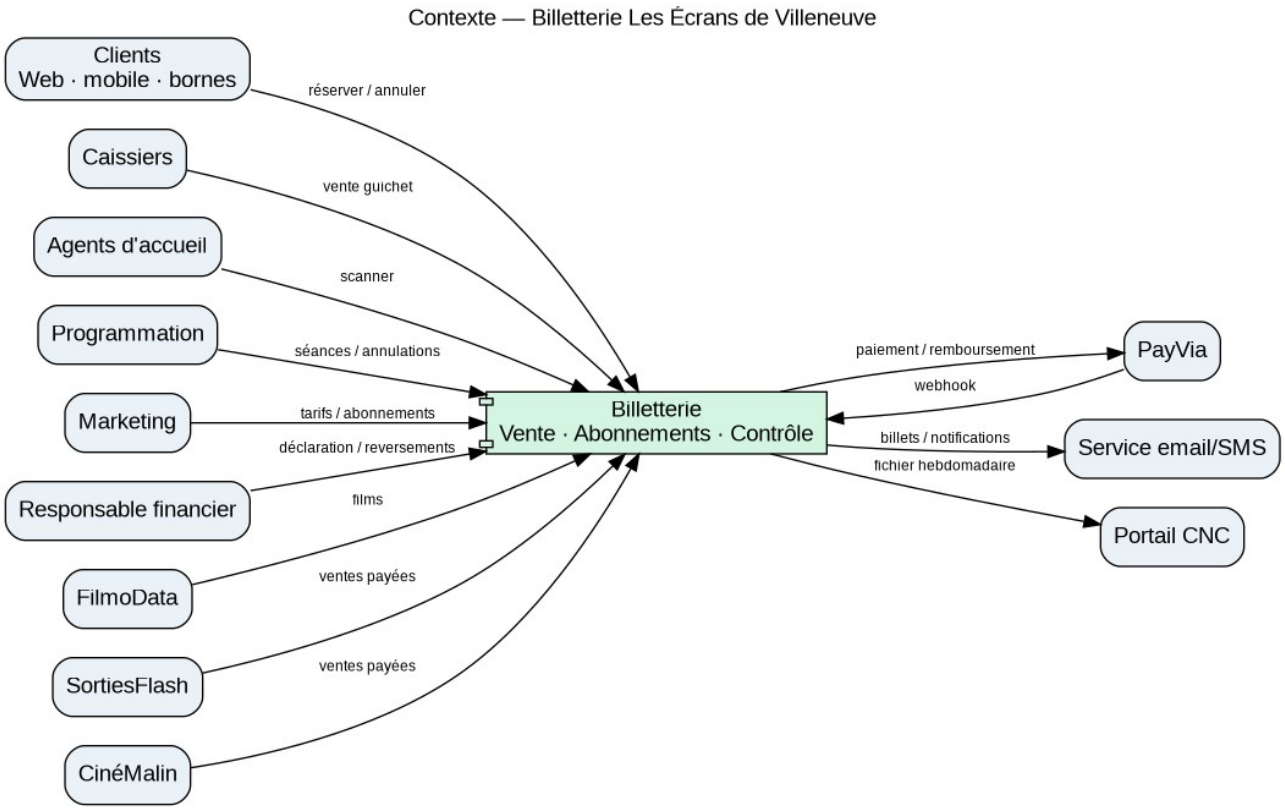
Code	Contrainte	Conséquence sur l'architecture
C1	Java 21, Spring Boot, PostgreSQL ; équipe de deux développeurs	Technologies imposées ; code métier Java pur, Spring dans les adapters
C2	Hébergement cloud en France, RGPD	Hébergement UE/France, minimisation, chiffrement, rétention 3 ans puis anonymisation
C3	PayVia imposé pendant 4 ans	Port Paiements et ACL WebhookPayVia isolent le contrat externe
C4	Déclaration CNC hebdomadaire avant jeudi midi	Événements et données de vente conservés ; export au format CNC
C5	Bornes en page web plein écran	API REST commune au site, mobile et bornes
C6	Mise en service avant fin 2027	Migration progressive et tests de non-régression

Hypothèses sur le brief

Question / zone grise	Hypothèse retenue	Justification
Annulation d'une séance : billet partenaire	Le cinéma rembourse le partenaire via un règlement mensuel compensatoire ; le partenaire rembourse son client final	Le paiement initial a été encaissé par le partenaire ; nous ne détenons pas son payment_id. Accord contractuel à formaliser
Résiliation abonnement et réservations du mois suivant	Les réservations déjà confirmées restent valables jusqu'à leur séance ; aucune nouvelle réservation gratuite après la date d'effet	Évite de retirer un droit déjà promis ; cohérent avec résiliation effective en fin de mois
Espèces au guichet : blocage de 10 min ?	Oui, même cycle de réservation ; le caissier confirme immédiatement le paiement espèces	Un seul modèle de cohérence tous canaux ; le blocage est généralement très bref
Douchettes hors ligne : durée et données	Autonomie cible de 4 h avec manifeste signé des billets de la journée ; synchronisation à la reconnexion	Permet un incident réseau sur avant-première ; le risque de double scan hors ligne est accepté comme décrit en B12
Paiement reçu après expiration	Appliquer B13 : conclure si les places sont encore libres, sinon déclencher un remboursement	Le sujet décrit explicitement cette règle ; traitement idempotent à renforcer
Données CNC	Le prix déclaré est le montant effectivement payé par le spectateur ; pour un partenaire, conserver son montant payé	Précision 3.4 et B11

3. Contexte

Le système étudié est la Billetterie. Les domaines hors périmètre (bar, confiserie, publicité, groupes/scolaires, paie et comptabilité) sont représentés uniquement comme voisins éventuels.



Les acteurs humains sont les clients, caissiers, agents d'accueil, programmateur, marketing et responsable financier. Les systèmes externes sont FilmoData, PayVia, SortiesFlash, CinéMalin, le prestataire email/SMS et le portail CNC.

4. Domaine

Glossaire

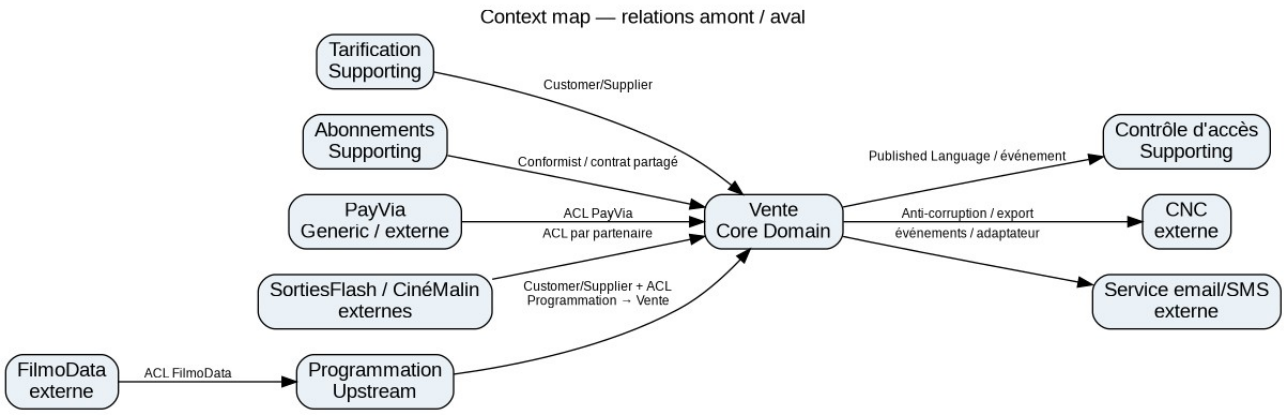
Terme	Contexte	Définition
Séance	Programmation	Projection planifiée : film, salle et horaire.
Séance	Vente	Plan de salle commercialisé : ensemble des fauteuils avec leur état libre, bloqué ou vendu.
Place / fauteuil	Vente	Emplacement numéroté dans une salle, identifié par PlaceId.
Réservation	Vente	Blocage temporaire d'une ou plusieurs places pendant 10 minutes ; ce n'est pas encore une vente.
Commande	Vente	Ensemble de 1 à 10 billets d'une même séance, suivis jusqu'au paiement/expiration.
Billet	Vente	Droit d'accès associé à une place et à un tarif.
Blocage	Vente	État temporaire d'une place attribuée à une commande et expirant à une heure précise.
Vente	Vente	Transformation définitive d'un fauteuil en état vendu après paiement confirmé.
Tarif	Tarification / Vente	Catégorie et prix calculé au moment du choix, figé dans le billet.
Opération	Tarification	Prix promotionnel qui remplace la grille pendant sa période.

Abonné	Abonnements	Client ayant une formule active et un droit à une place gratuite par séance.
No-show	Abonnements	Billet gratuit d'abonné non scanné à la fin de sa fenêtre de validité.
Scan	Contrôle d'accès	Lecture du QR code qui consomme le droit d'entrée une fois.
Vente partenaire	Vente	Vente déjà payée chez un tiers, transmise à la billetterie dans le format du partenaire.
Fenêtre de validité	Contrôle d'accès	De 30 minutes avant à 15 minutes après le début de la séance (B12).
ACL	Architecture	Couche anti-corruption qui traduit le modèle d'un système externe vers notre langage.

Sous-domaines

Sous-domaine	Type	Stratégie
Vente de places	Core domain	DDD tactique : agrégats Seance et Commande, invariants, événements ; différenciation métier et risque de double vente
Abonnements	Supporting	Architecture simple / CRUD paramétré ; règles documentées, sans DDD complet
Tarification	Supporting	Modèle de règles et dates d'effet ; évolue souvent avec les opérations marketing
Programmation	Supporting	CRUD métier maîtrisé par le programmateur ; ACL FilmoData
Contrôle d'accès	Supporting	Lecture rapide du billet et scan hors ligne ; stockage local synchronisable
Déclaration CNC	Generic / réglementaire	Adaptateur d'export vers format imposé
Paiement	Generic	PayVia externe isolé derrière port et ACL
Notification email/SMS	Generic	Prestataire du marché derrière adaptateur

Context map



Amont (U)	Aval (D)	Pattern	Pourquoi
Programmation	Vente	Customer/Supplier + ACL	La programmation publie ses séances ; Vente traduit son modèle en SeanceId/plan commercial
Tarification	Vente	Customer/Supplier	Vente consomme les prix via un port ; prix figé dans le billet

Abonnements	Vente	Conformist (contrat explicite)	Vente applique le droit gratuit fourni par Abonnements
FilmoData	Programmation	ACL	API anglaise et durées en secondes isolées
PayVia	Vente	ACL entrante/sortante	Webhook et paiement traduits dans notre langage
SortiesFlash	Vente	ACL dédiée	Format imposé, ne doit pas fuiter hors de l'adapter
CinéMalin	Vente	ACL dédiée à créer	Même stratégie que SortiesFlash, mapping différent
Vente	Contrôle d'accès	Published Language / événements	CommandePayee alimente les billets contrôlables
Vente	CNC	Adaptateur d'export	Format réglementaire imposé
Vente	Service email/SMS	Adaptateur / événements	Envoi de billets après paiement

5. Choix d'architecture par contexte

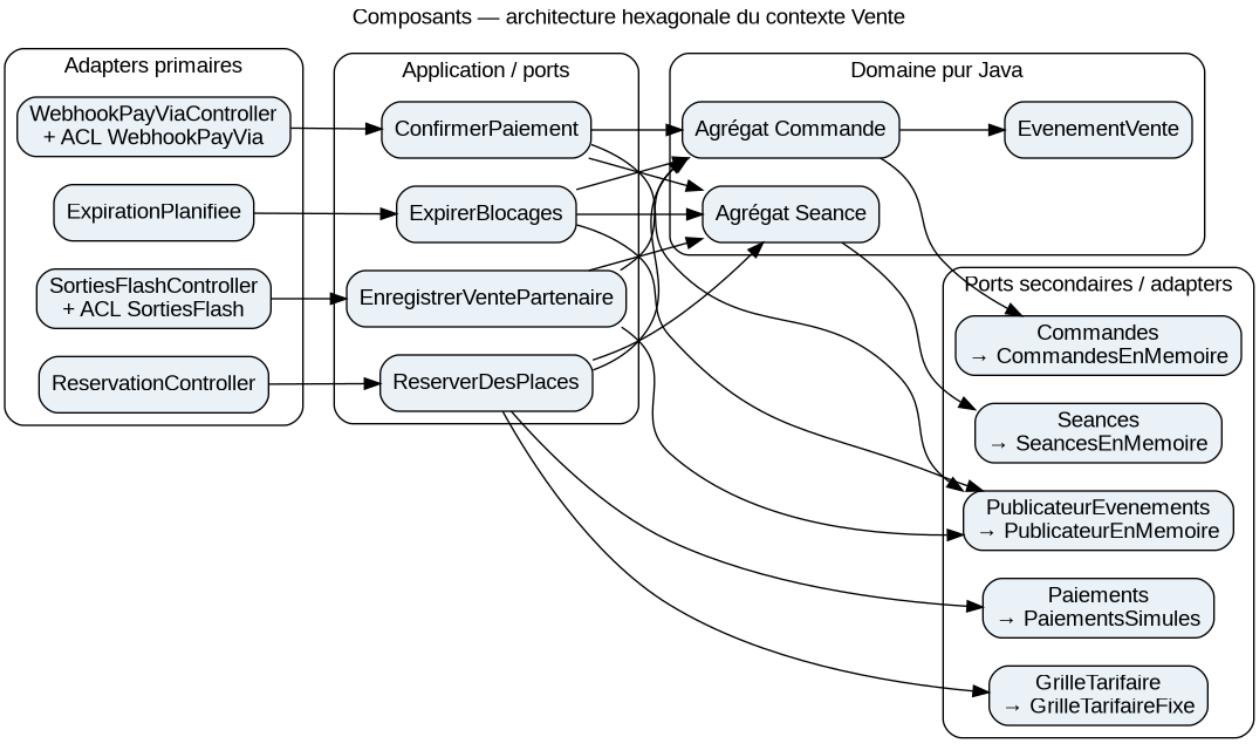
Échelle de décision proposée : complexité, valeur métier, durée de vie, volatilité, disponibilité des experts et nombre de canaux sont notés de 1 (faible) à 5 (fort). Les notes sont une appréciation argumentée de l'équipe, à confirmer en atelier.

Contexte	Complexité	Valeur	Durée	Volatilité	Experts	Canaux	Total	Architecture retenue
Vente	5	5	5	4	4	5	28	DDD + hexagonale
Abonnements	4	4	4	5	3	3	23	CRUD paramétré + services ; pas de DDD complet
Tarification	3	4	4	5	3	4	23	DDD léger + ports
Programmation	2	3	4	2	2	2	15	CRUD + ACL FilmoData
Contrôle d'accès	4	4	4	3	4	2	21	Modèle métier ciblé + edge offline
Déclaration CNC	3	2	3	2	2	1	13	Batch / adaptateur
Paiement	2	2	2	2	1	2	11	Intégration générique via port

Justification

Le DDD tactique est concentré sur Vente : l'invariant B1 est critique, la charge peut atteindre 420 places en moins de cinq minutes et plusieurs canaux concurrents partagent le même stock. Une architecture hexagonale permet de remplacer les adapters (stockage, paiement, API) sans réécrire les règles. À l'inverse, Programmation et CNC n'ont pas besoin d'un modèle riche dans ce périmètre : un CRUD et des adaptateurs suffisent.

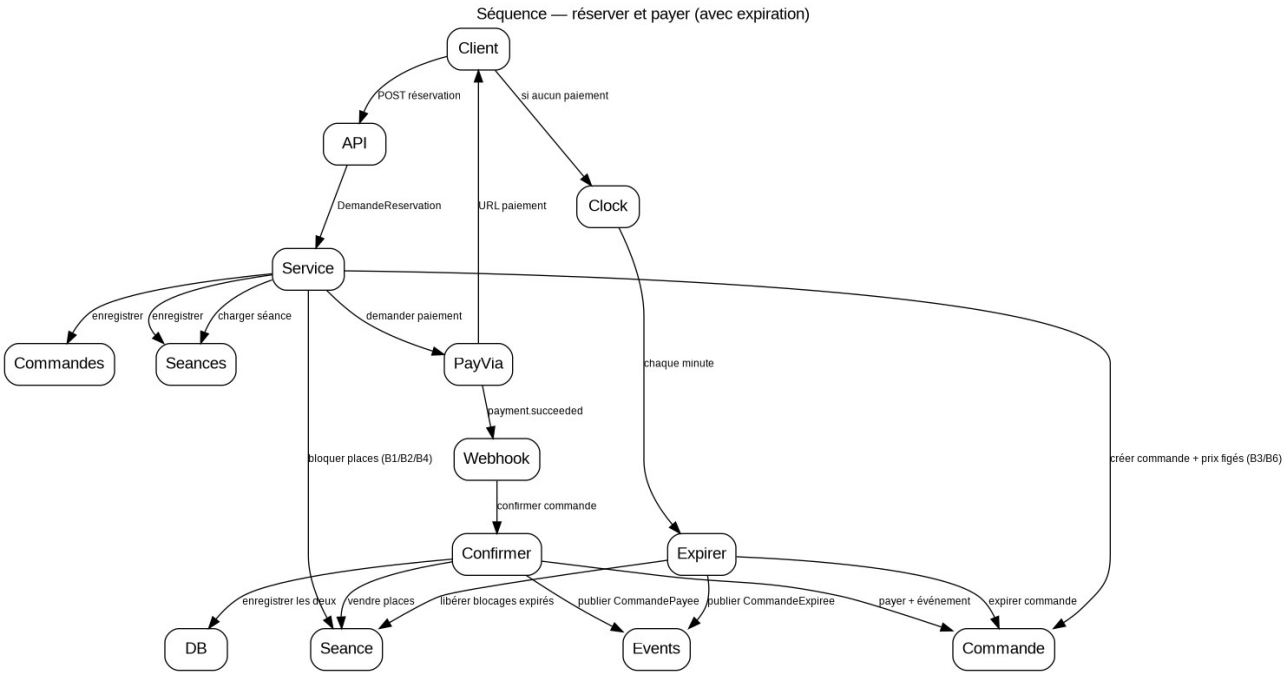
6. Vue composants



Les adapters primaires REST, webhook PayVia, planificateur et SortiesFlash appellent les ports entrants. Les services orchestrent les ports sortants. Le domaine ne dépend ni de Spring, ni des adapters, ni de la base. L'inversion de dépendance est matérialisée par les interfaces de application/port.

7. Vue dynamique

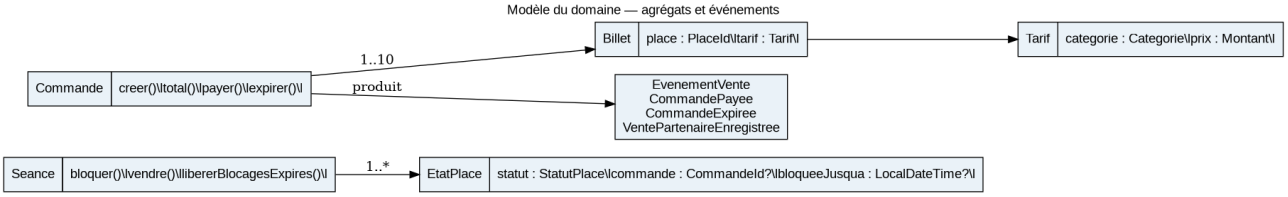
Réserver et payer des places



Le service de réservation calcule le prix, bloque les places, crée la commande et l'enregistre avant de demander l'URL de paiement. PayVia appelle ensuite le webhook. La confirmation charge commande et séance, vend les places et passe la commande à PAYEE avant d'enregistrer les deux. En l'absence de paiement, le planificateur libère les blocages expirés et fait expirer les commandes en attente.

8. Modèle du domaine

Agrégats et value objects



Seance est l'agrégat garantissant la disponibilité d'un fauteuil : sa map PlaceId → EtatPlace porte l'invariant de non-double-vente. Commande porte le cycle de vie d'un achat et la liste immuable des billets. Les deux agrégats sont liés par SeanceId, pas par une référence objet.

Value objects : SeanceId, CommandeId, PlaceId, Montant (centimes), Tarif (catégorie + prix figé), Billet, EtatPlace. Les identifiants évitent de manipuler des chaînes non validées ; Montant en centimes évite les erreurs d'arrondi.

Invariant → agrégat → test

Invariant (règle)	Agrégat qui le protège	Test qui le prouve
B1 : une place vendue une seule fois	Seance	SeanceTest / réservation concurrente et vente partenaire
B2 : blocage de 10 minutes puis libération	Seance + Commande	SeanceTest ; ExpirerBlocagesTest
B3 : maximum 10 places par commande	Commande	CommandeTest.refuse_plus_de_10_places_B3
B4 : vente fermée à H+15	Seance	SeanceTest
B6 : prix garanti pendant le paiement	Commande / Billet	CommandeTest.total ; ReserverDesPlacesTest
B13 : paiement tardif si place libre, sinon remboursement	Seance + Commande	PaiementTardifTest (bonus)

Événements et policies

Les agrégats enregistrent les événements sans les publier eux-mêmes. Après persistance, l'application publie CommandePayee, CommandeExpiree et VentePartenaireEnregistree. Les policies futures consomment ces événements pour envoyer les billets, alimenter le contrôle d'accès, notifier le client, établir le CNC et calculer les reversements.

9. Décisions (ADR)

ADR-0001 — Architecture retenue par contexte

Rubrique	Contenu
Statut	Accepté — à valider par l'équipe
Contexte	La billetterie doit durer au moins dix ans, accueillir plusieurs canaux et être maintenable par deux développeurs internes.
Options envisagées	Décision 1 — Tarification : A. contexte Tarification séparé (modèle et expert marketing dédiés) ; B. calcul

	intégré à Vente. Décision 2 — Abonnements : A. DDD complet (agrégat et événements) ; B. architecture simple avec services, requêtes et formules paramétrées.
Décision	Décision 1 : option A, Tarification est un bounded context supporting distinct ; Vente consomme le port GrilleTarifaire. Décision 2 : option B, Abonnements reste volontairement simple et paramétrable, car Sabine n'est disponible qu'une fois par mois et les formules changent souvent. Les règles restent explicites et testées, mais pas de DDD complet à ce stade.
Conséquences positives	Tarifs et opérations évoluent sans grossir le Core Domain Vente ; les formules peuvent changer rapidement sans maintenir un agrégat complexe construit avec peu d'accès à l'experte.
Conséquences négatives	Une relation de contexte supplémentaire et un appel de tarification à surveiller ; les règles d'abonnement en services/requêtes peuvent se disperser et devront être refactorées si le domaine devient plus complexe.

ADR-0002 — Agrégats et concurrence de la vente

Rubrique	Contenu
Statut	Accepté — à valider par l'équipe
Contexte	B1 interdit toute double vente ; un pic de 420 places peut arriver en moins de cinq minutes. Le paiement est asynchrone et les blocages expirent.
Options envisagées	Frontières : A. agrégat unique Billetterie ; B. deux agrégats Seance et Commande. Concurrence (décision 3) : A. verrou optimiste sur toute la séance ; B. verrouillage par place avec contrainte unique (séance, place) en base.
Décision	Deux agrégats (option B) : Seance protège les états des fauteuils, Commande protège le cycle de paiement et référence Seance par identifiant. Pour la production, décision 3 : option B, verrouiller par place et imposer une unicité (seance_id, place_id) en base dans une transaction ; prendre les verrous dans l'ordre des PlaceId pour limiter les interblocages. Le squelette pédagogique conserve le verrou optimiste par séance, plus simple à démontrer, mais il provoquerait trop de conflits inutiles pendant un pic de 420 places.
Conséquences positives	Deux clients achetant des places différentes ne se bloquent pas ; B1 est garanti par la contrainte de base même si plusieurs canaux écrivent simultanément ; agrégats de taille maîtrisée.
Conséquences négatives	La règle B1 est en partie portée par la base et non seulement par l'agrégat ; réservation multi-places plus complexe, ordre de verrouillage indispensable et transaction à tester sous charge.

10. Qualité et tests

La stratégie suit la pyramide : tests unitaires rapides du domaine, tests de services avec adapters en mémoire et horloge contrôlable, puis tests d'intégration HTTP Spring MockMvc. Les tests fournis ne doivent jamais être modifiés.

Niveau	Ce qu'on teste	Comment
Domaine	Invariants B1 à B6, transitions de commande, tout-ou-rien	JUnit 5, tests SeanceTest et CommandeTest
Application	Orchestration réservation, paiement, expiration et vente partenaire	JUnit 5 avec adapters en mémoire et Clock fixe
Intégration	Contrats HTTP 201/204/400/404/409/422 et JSON partenaires	Spring Boot Test + MockMvc ; ApiVenteTest et VentePartenaireApiTest

Règles de dépendance : domaine → Java uniquement ; application → domaine et ports ; adapters → application/domaine ; configuration → adapters et services. L'heure est injectée via Clock ; les services n'appellent pas directement LocalDateTime.now() sans horloge.

11. Risques et dette technique

Risque ou simplification	Impact	Parade
Concurrence forte au lancement des avant-premières	Conflits de réservation et pression base de données	Verrou optimiste/version, transaction atomique, test de charge et stratégie de retry
Douchettes hors ligne	Double scan possible pendant une coupure	Manifeste signé et limité dans le temps, synchronisation et journal local ; risque résiduel assumé B12
Paiement webhook dupliqué (B14)	Double confirmation ou double remboursement	Clé d'idempotence durable par événement/payment_id ; tests de répétition
Stockage mémoire de la version pédagogique	Perte des données au redémarrage et absence de transaction DB réelle	Remplacer par PostgreSQL/JPA ; ajouter outbox transactionnelle
CNC / partenaires non encore contractualisés	Écart de format ou de responsabilité de remboursement	Tests de contrat, ACL séparées et validation juridique
RGPD abonnés	Conservation excessive de données personnelles	Durée de conservation de 3 ans après fin d'abonnement, anonymisation, accès limité et journalisation

Annexe — état d'implémentation

Le code livré complète le cœur des étapes 1 à 5, corrige la dépendance vers l'adapter de tarification (étape 6), branche les endpoints Spring de l'étape 7 et ajoute le cas d'usage/ACL SortiesFlash de l'étape 8. Les bonus (interface de réservation, plan consultable, persistance JPA complète, annulation de séance) ne sont pas inclus dans le périmètre obligatoire. La validation finale des tests Spring nécessite Maven et un accès aux dépendances du pom.

Justification des scores : Vente obtient le score maximal car B1, les cinq canaux, le pic des avant-premières et la durée de vie dix ans concentrent valeur et complexité. Tarification et Abonnements sont volatils ; Abonnements reste simple faute d'accès fréquent à l'experte. Programmation est stable et dispose d'un expert peu disponible ; elle peut rester CRUD. Le CNC est réglementaire mais peu différenciant ; un adaptateur suffit.